

The consensus problem in distributed systems

The consensus problem

- There are **N** nodes in the system
- Each node starts with input $\{0,1\}$
- The network is asynchronous but reliable
 - Messages can take arbitrarily long to be delivered
- Nodes operate at arbitrary speed, may fail by stopping (crash failure), and may restart.
- At most **1** node fails
- Goal: all nodes decide same value **v**, where v was an input

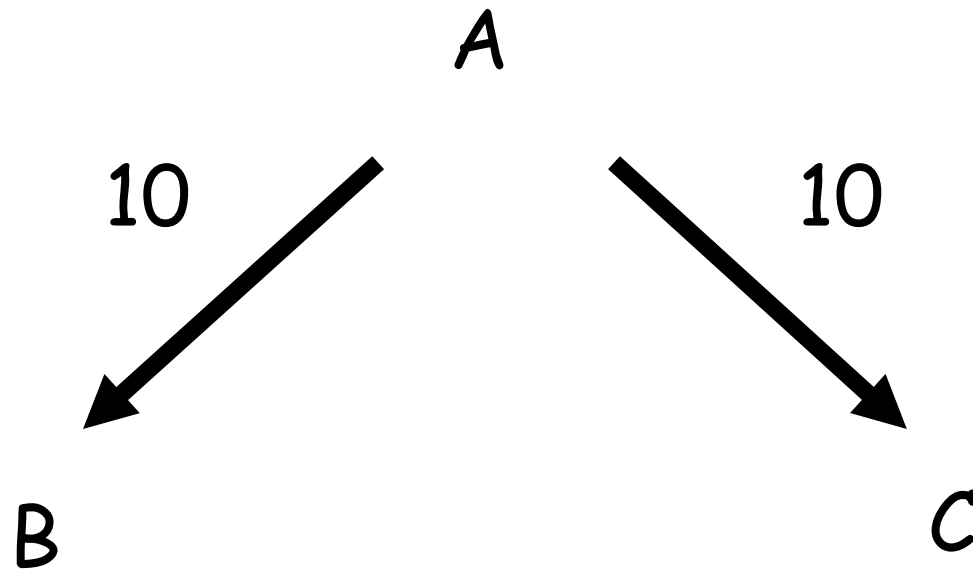
Fault-tolerant consensus protocol ^{TODO}

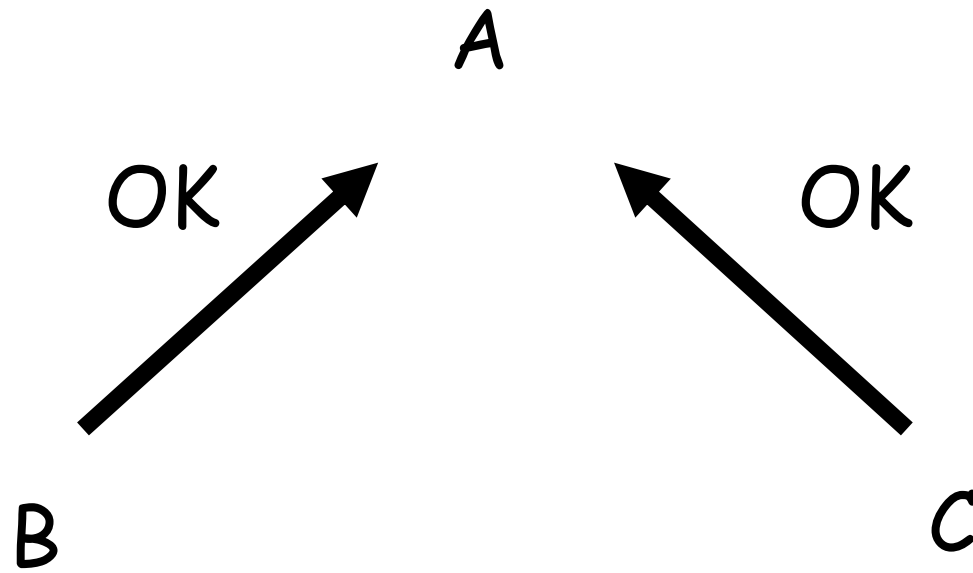
- **Collect votes** from all N nodes
- Wait for the **majority** of nodes to respond, and tell everyone the outcome (choose value for the output)
- Nodes "**decide**" (i.e. they accept the outcome)
- There is a **problem** if a message is delayed or a node restarts after a failure.

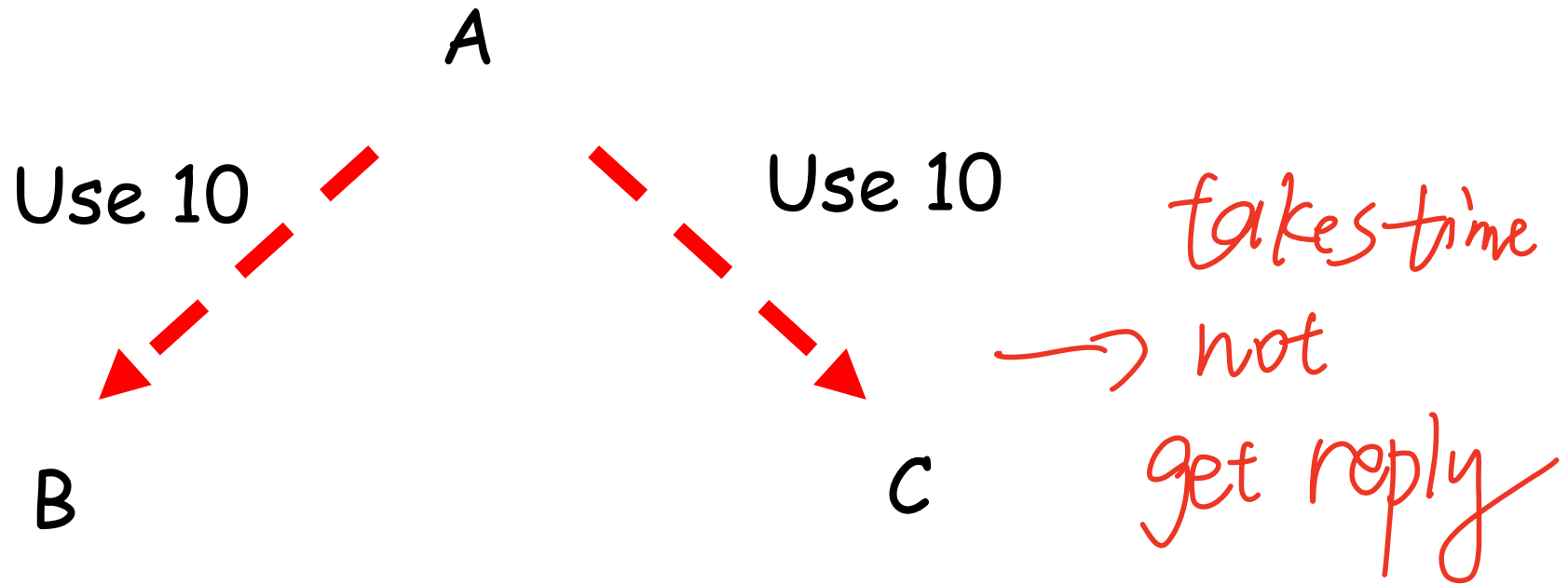
A

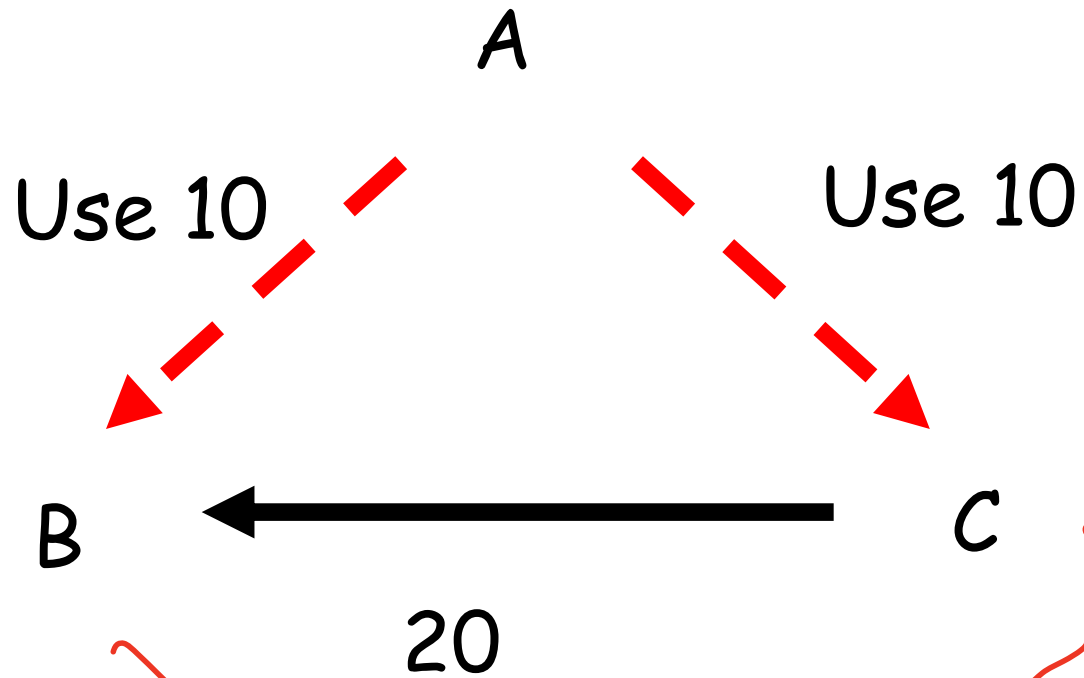
B

C

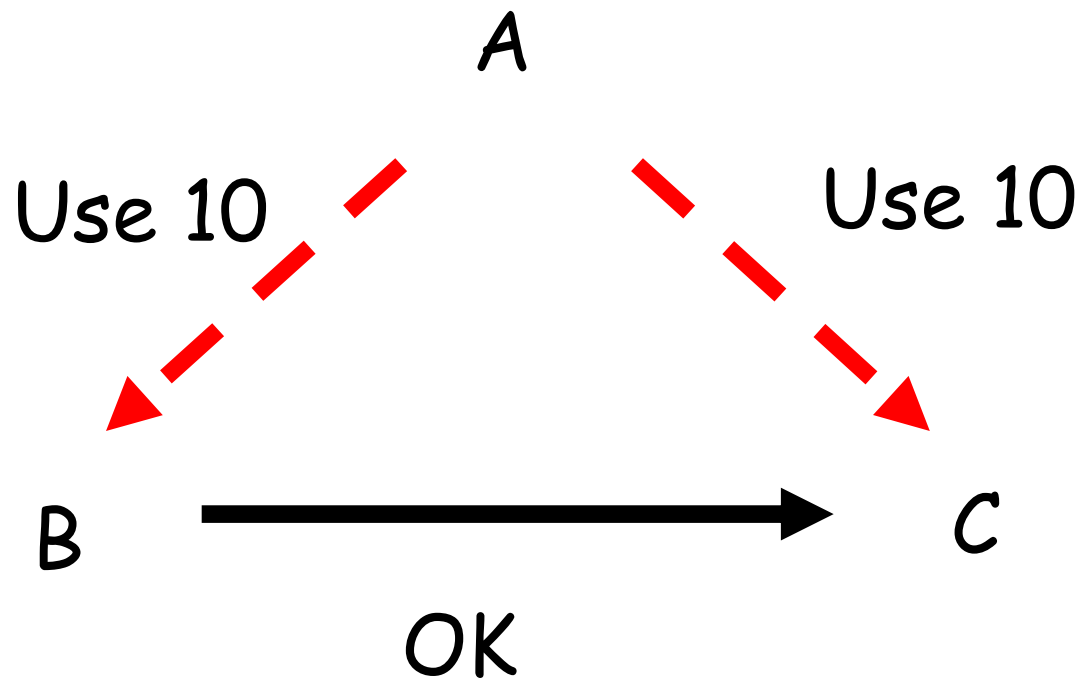


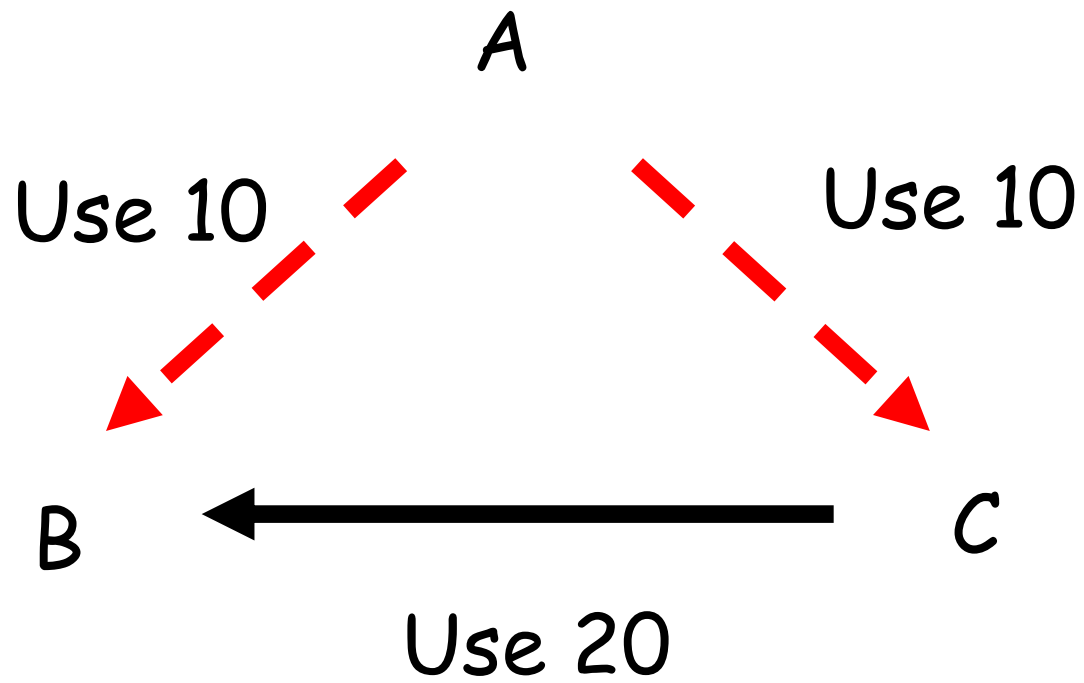


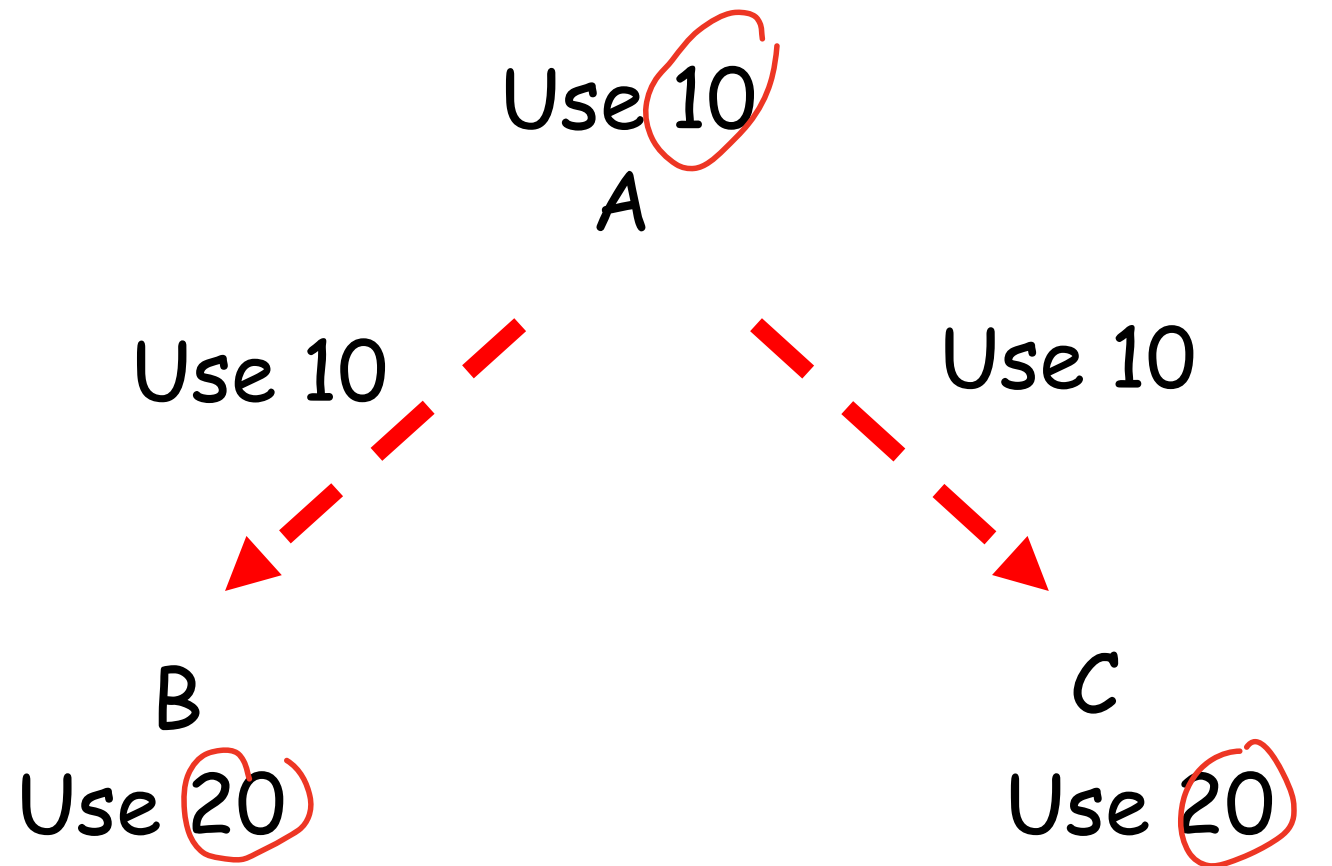




assume
A crash.







FLP Impossibility of Consensus

- A surprising result
 - In an asynchronous model where only one node might crash, there is no fault-tolerant distributed algorithm that solves the consensus problem.
- They prove that no consensus algorithm is guaranteed to terminate in the presence of crash faults
 - This is true even if no crash actually occurs
 - Proof constructs infinite non-terminating runs

Intuition of FLP

1. A system tries to agree on which command to execute next
2. Node p's messages are delayed during the transmission
 - p is regarded as failed
3. Since the system is fault-tolerant, if p crashes, the system should adapt and move on to reach a decision
4. Before the decision is finally reached, p's messages arrive. So, p has to be included in decision making.
5. We are back to the beginning (step 1).
 - This takes time and no real progress occurs between 1 and 4.

restart → delay → ¹³re
Start

The meaning of "impossibility"

- In formal proofs, an algorithm is totally **correct** if
 - It is **safe**.
 - It always **terminates**.
- FLP proves that any fault-tolerant algorithm solving consensus has runs that **never terminate**.
 - These runs are extremely **unlikely** ("probability is close to zero")

Still use, chance ↗ ↘ 不停止. 概率小
- These runs mean that a totally correct solution for the consensus problem is **impossible**.
- It means consensus is **not always** possible.

Paxos Algorithm

- Distributed consensus algorithm
- Key Assumptions:
 - There are **n** nodes. The set of node is known a-priori.
 - Nodes suffer **crash** failures, nodes can **restart** after a failure
 - Network might be very **slow**
- Guarantees safety
 - Only a **single** value is chosen
 - Only a **proposed** value can be chosen
 - Once a value is chosen, it **remains** the only **chosen value**.
- Cannot guarantee **liveness**.

Details of Paxos

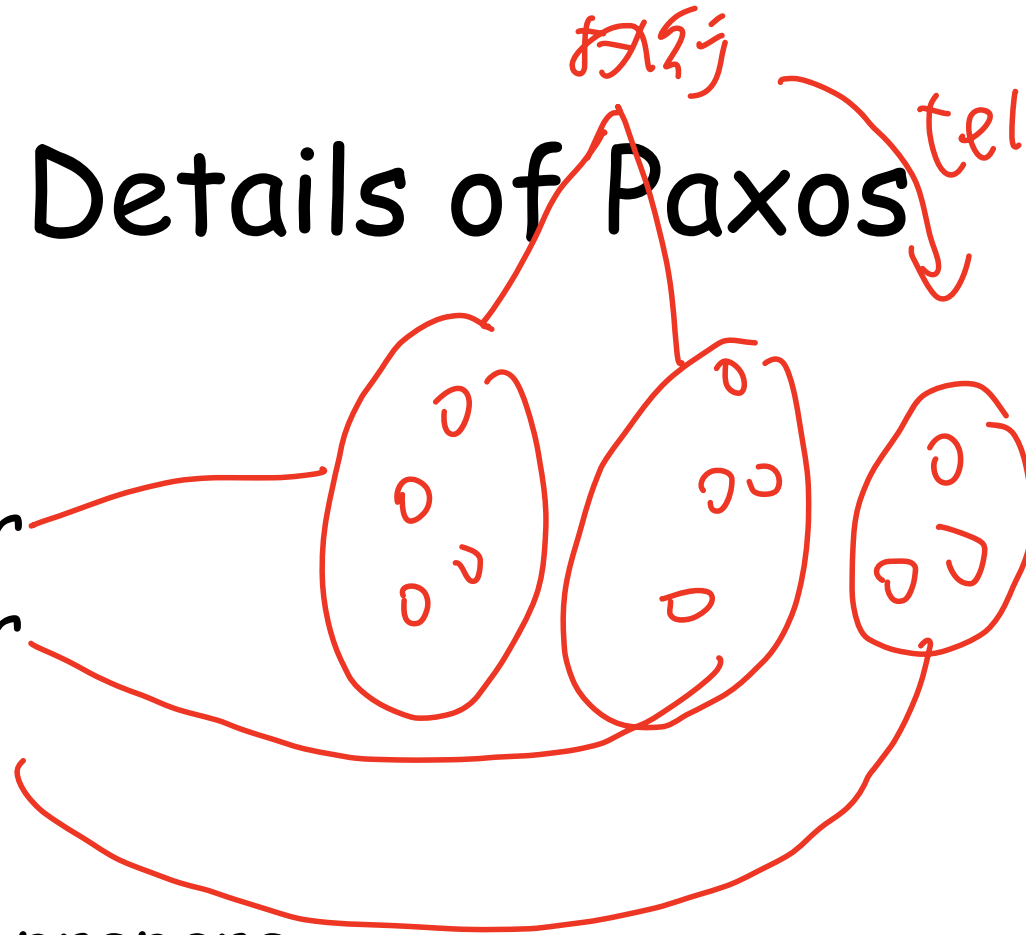
Handwritten notes in red: "telling learner value." with an arrow pointing from the title area to the right.

- 3 roles

- proposer
- acceptor
- learner

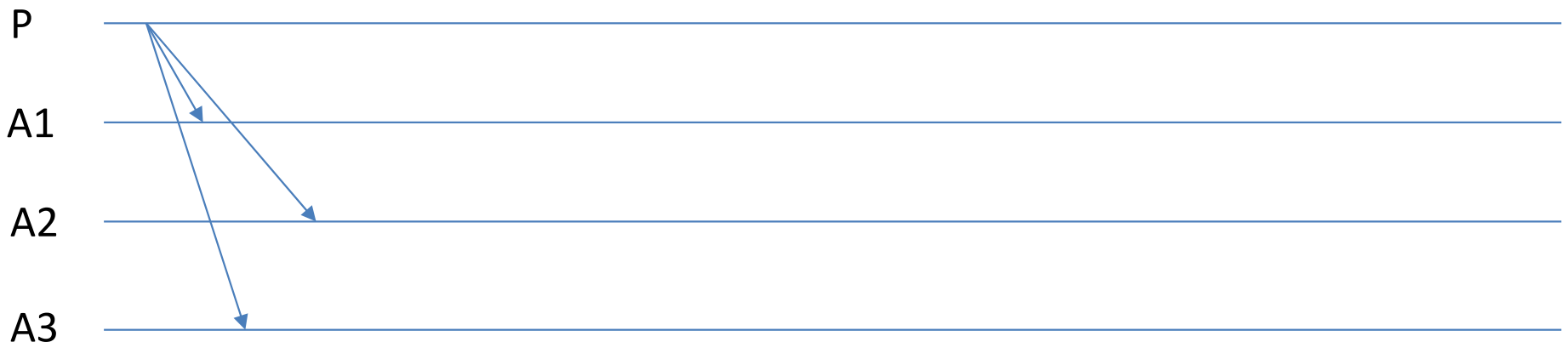
- 3 phases

- Phase 1: prepare
- Phase 2 (if get positive replies from a majority of the nodes): propose
- Phase 3: (if get positive replies from a majority of the nodes): finalise



Phase 1: (prepare request)

- A proposer chooses a new proposal version number n , and sends a prepare request ("prepare", n) to a majority of acceptors:
 - Can I make a proposal with version number n ?
 - If yes, do you suggest a value for my proposal?



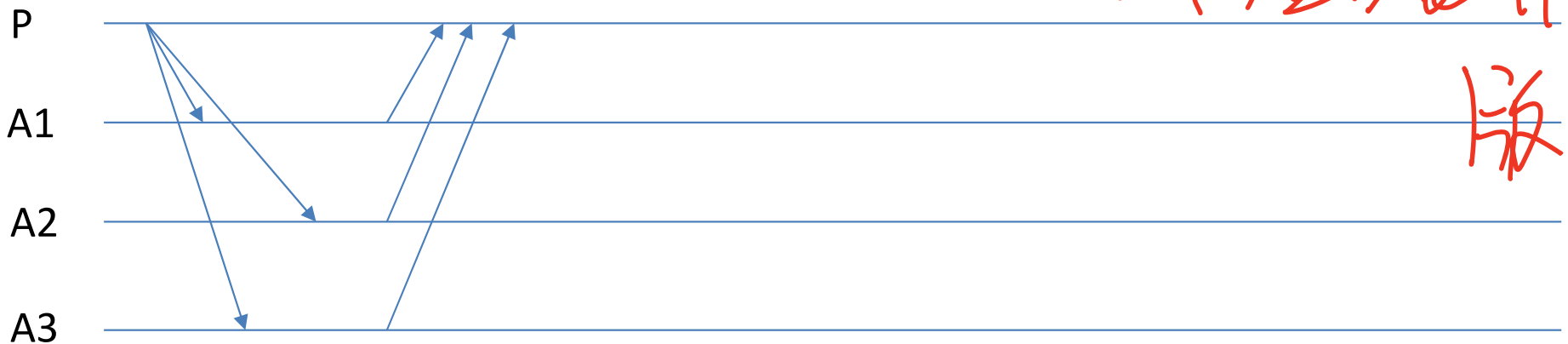
- When an acceptor receives a prepare request ("prepare", n) where n is greater than the version number of any prepare request the acceptor t has already responded, the acceptor sends out ("ack", n, n', v') or ("ack", $n, -, -$)
 - A respond is a **promises** not to accept any proposal with version number less than n .
 - A respond suggests the value v' of the **highest-number** proposal that the acceptor has accepted if any, else -
 - If the acceptor receives a prepare request with a larger version number, the acceptor **abandons** the engagement with the proposer with a smaller version number and engages the proposer with a larger version number.

value

$n < n'$

因大版本直接放弃小

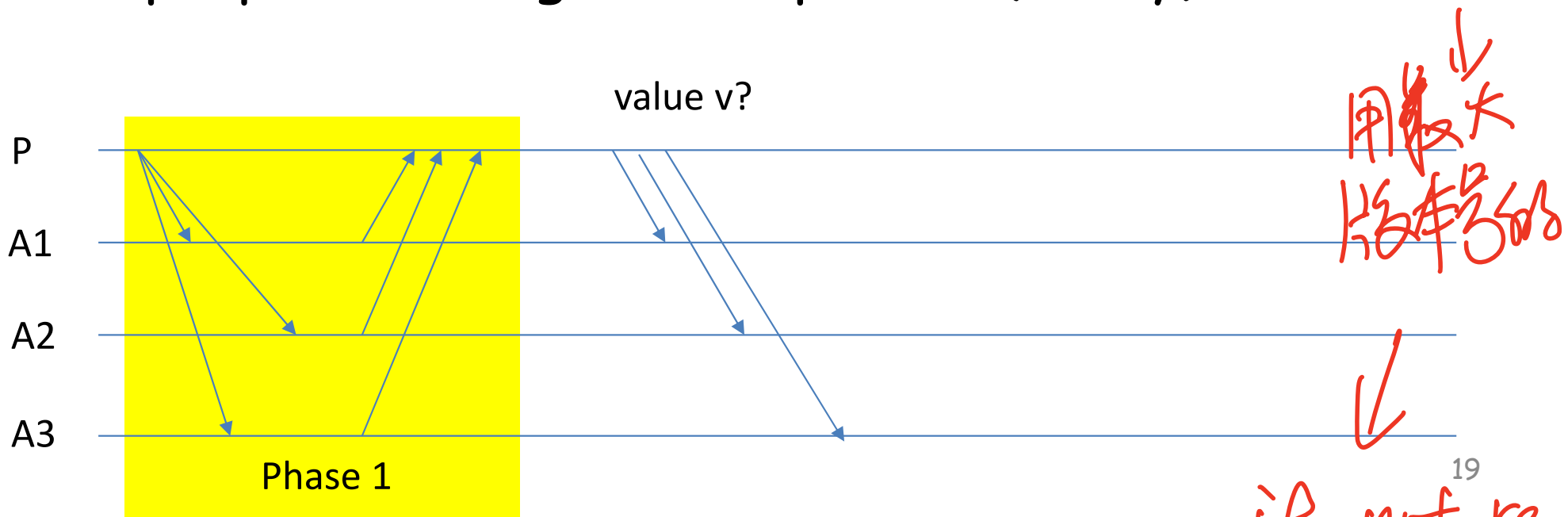
版本号



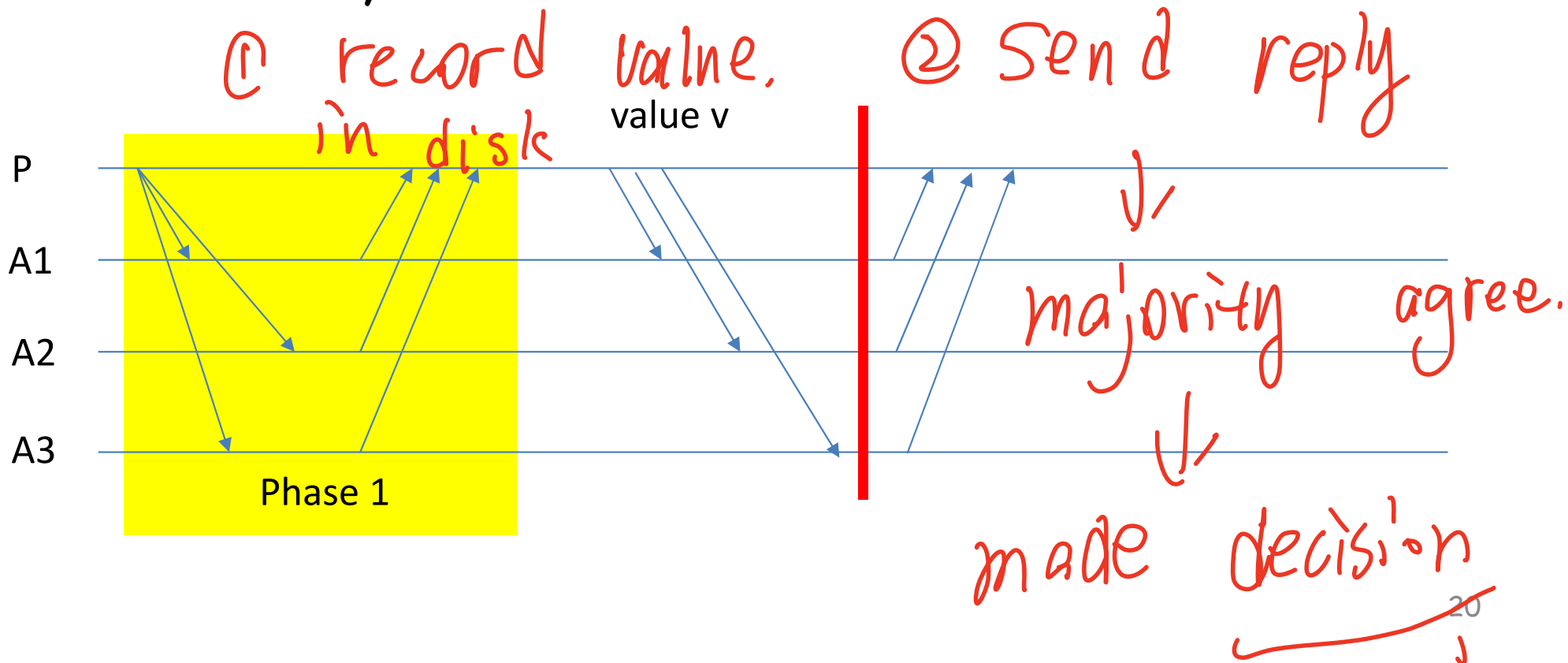
Phase 2: (accept request)

- If the proposer receives responses from a majority of the acceptors, it can issue a proposal for value (i.e. an accept request) ("accept", n , v) with version number n and value v :

- n is the number that appears in the prepare request.
 - v is the value of the highest version number proposal among the responses (if any)
- Some acceptor may tell with prepare message*

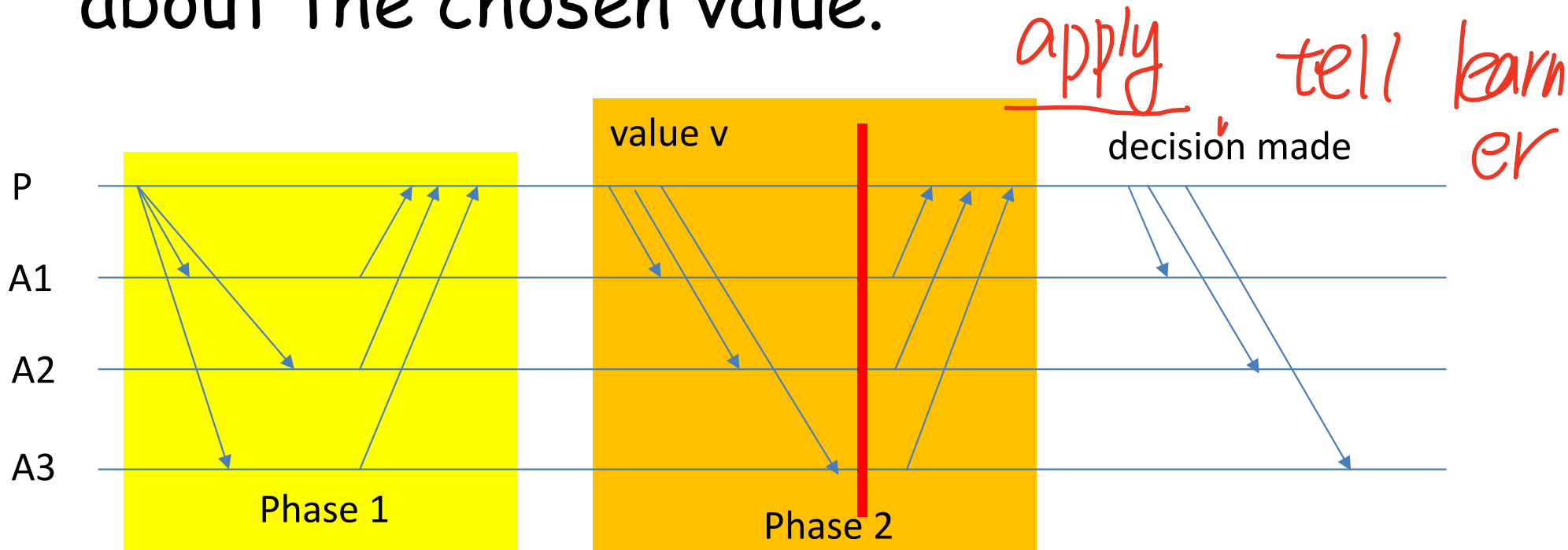


- If the acceptor receives an accept request ("accept", n , v), it accepts the proposal unless it has already responded to a prepare request having a version number greater than n .
 - The acceptor logs v on disk and sends ack to the proposer
- If a majority of the acceptors have logged v on their disk, the value is chosen.



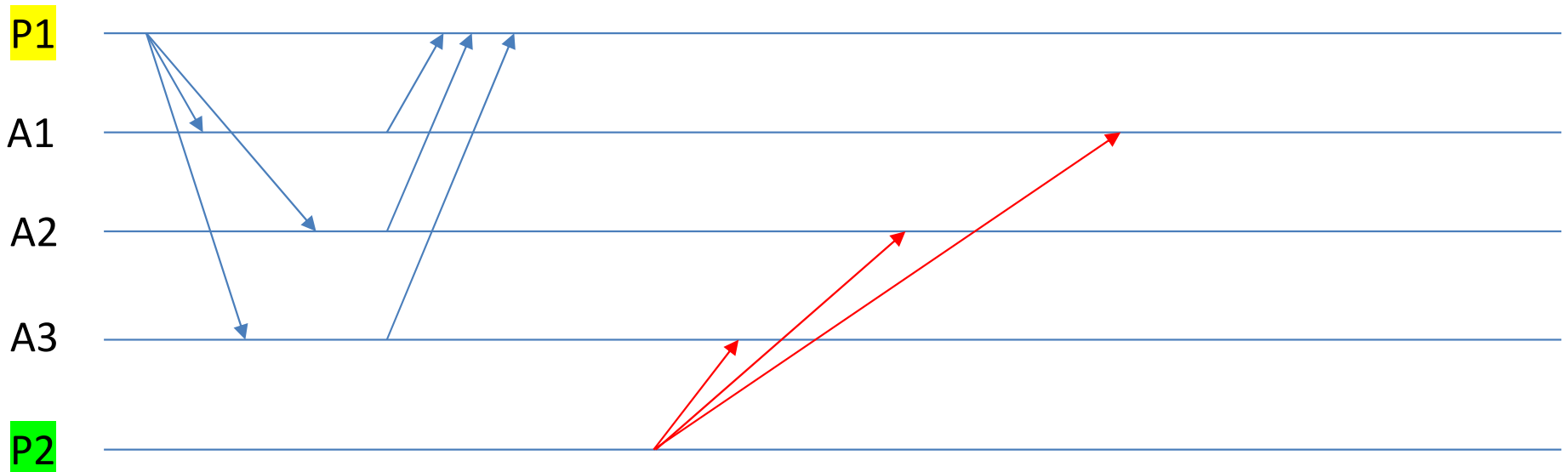
Phase 3 (finalise)

- If the proposer gets acknowledgments from a majority of the acceptors, it tells everyone about the chosen value.
- The acceptors can tell all the learners about the chosen value.



discussions

- If the acceptor receives a prepare request with a larger version number, the acceptor abandons the engagement with the proposer with a smaller version number and engages the proposer with a larger version number.
– why



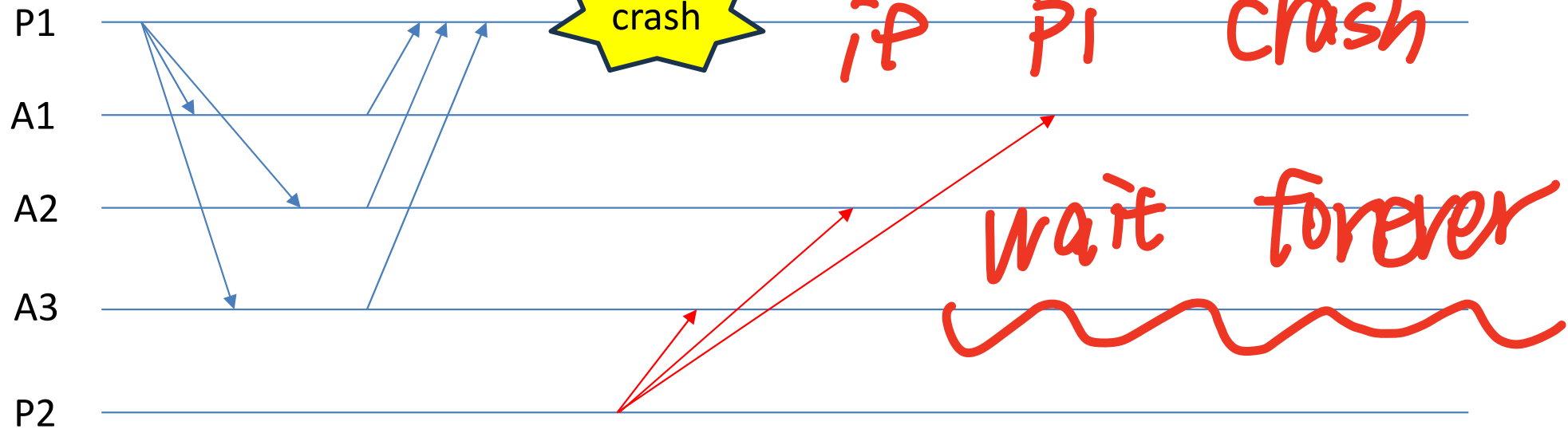
P_2 version number $> P_1$

12 version number

if not abandons

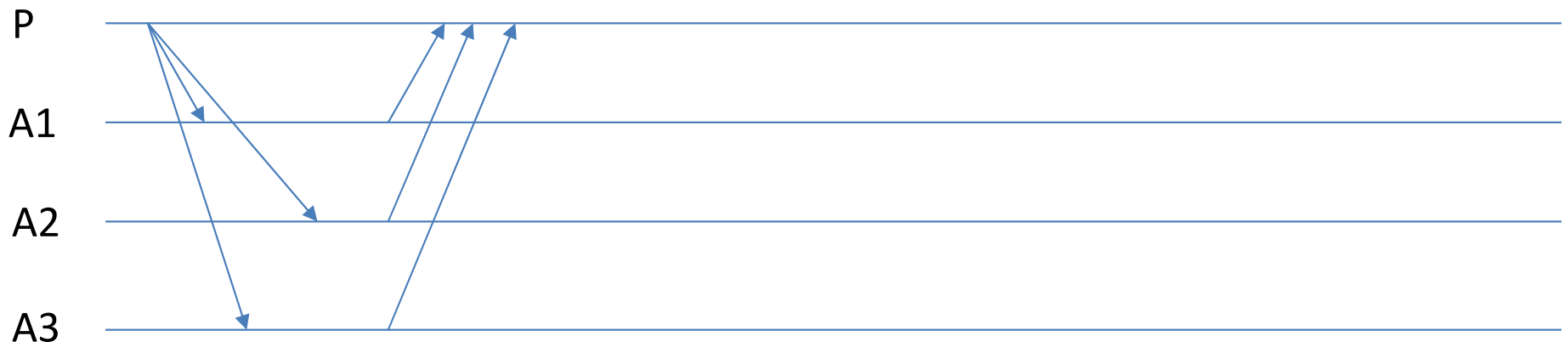
if wait for p1 reply

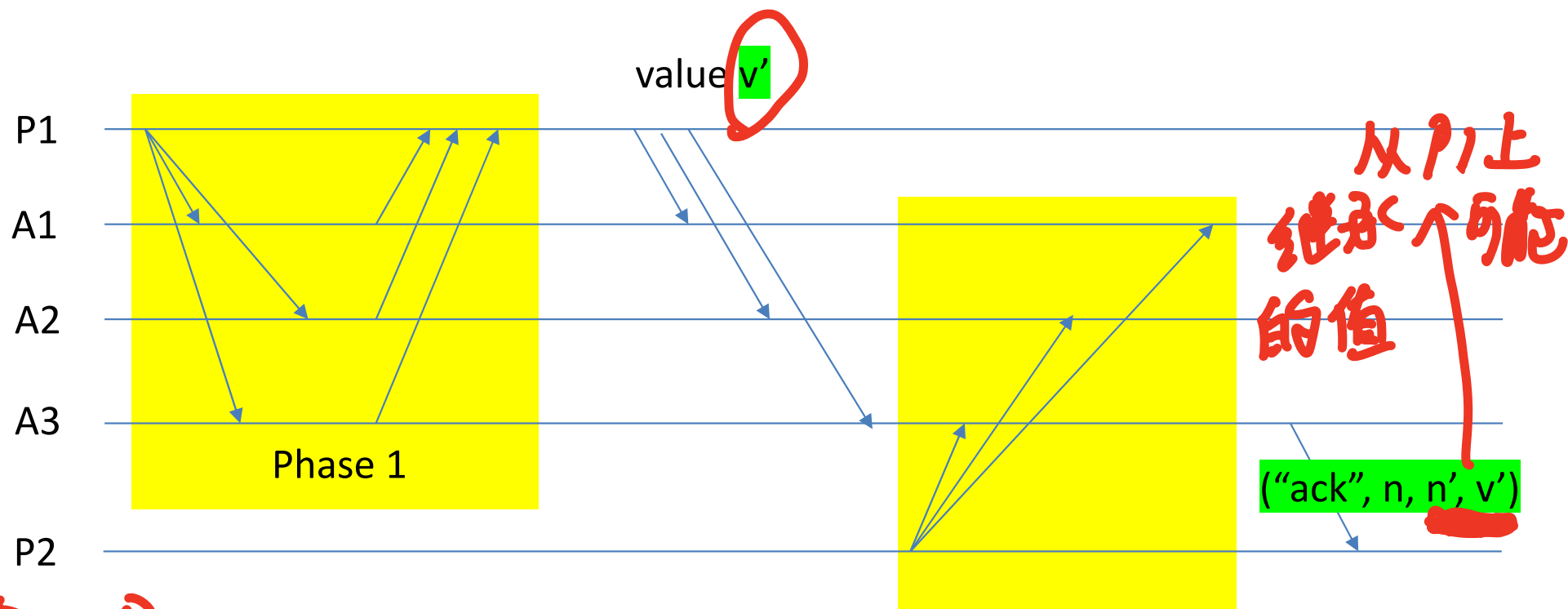
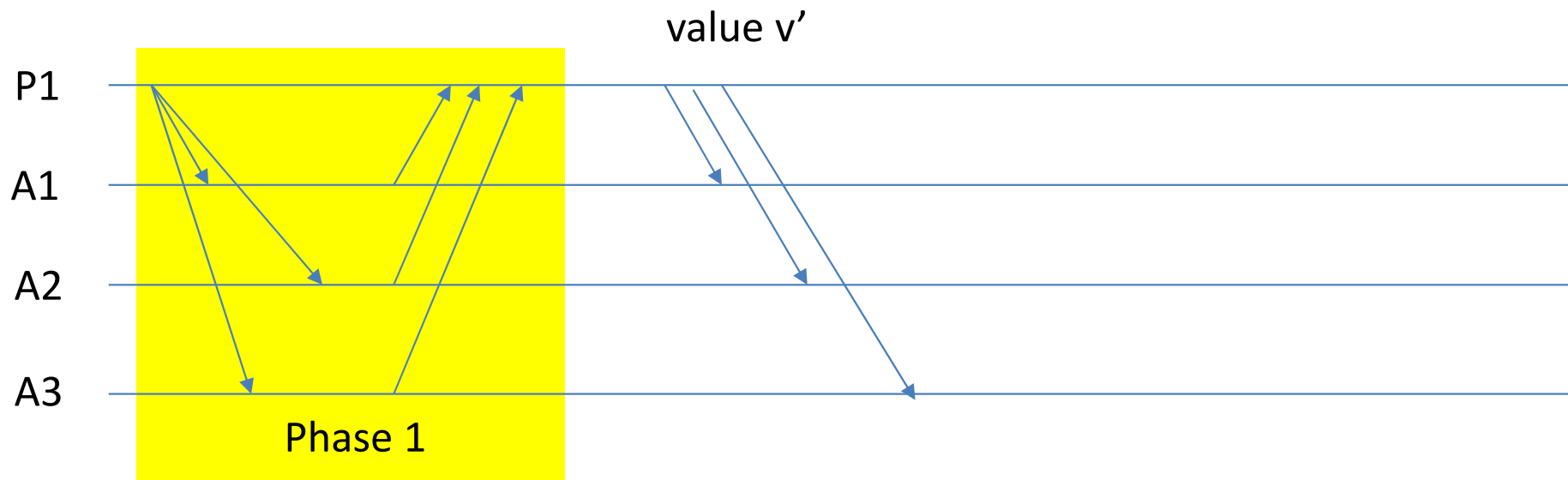
if p1 crash



discussions

- When an acceptor receives a prepare request ("prepare", n) where n is greater than the version number of any prepare request the acceptor t has already responded, the acceptor sends out ("ack", n , n' , v') or ("ack", n , -, -)
 - What are n' and v' ?

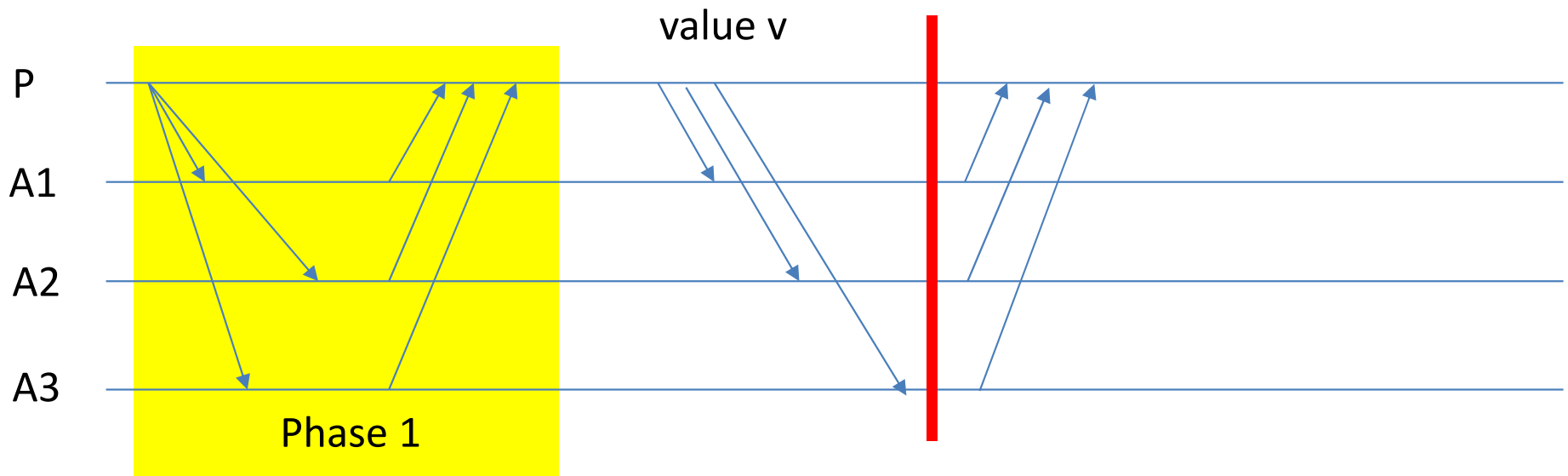


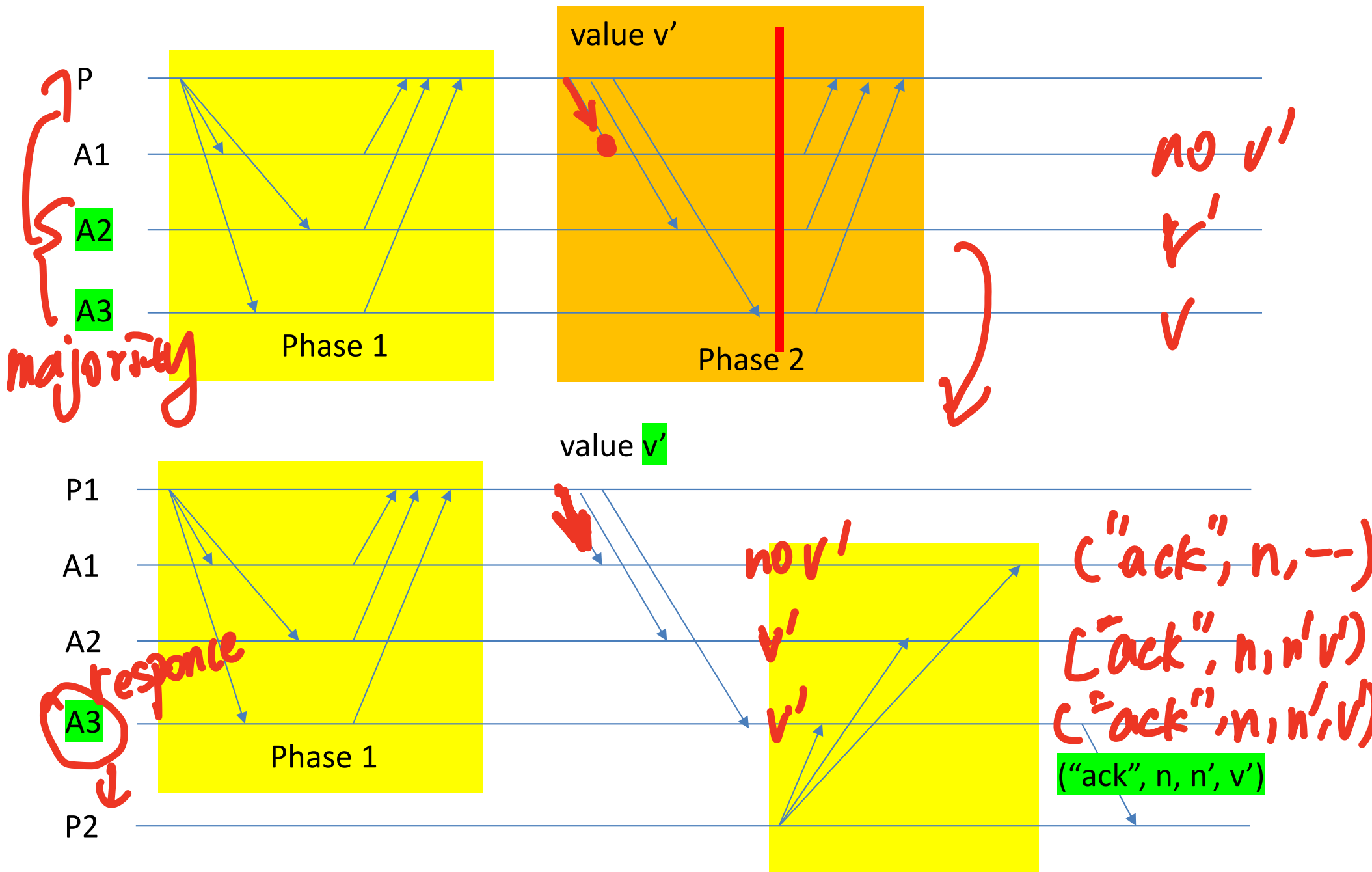


$P_2 > P_1$
version

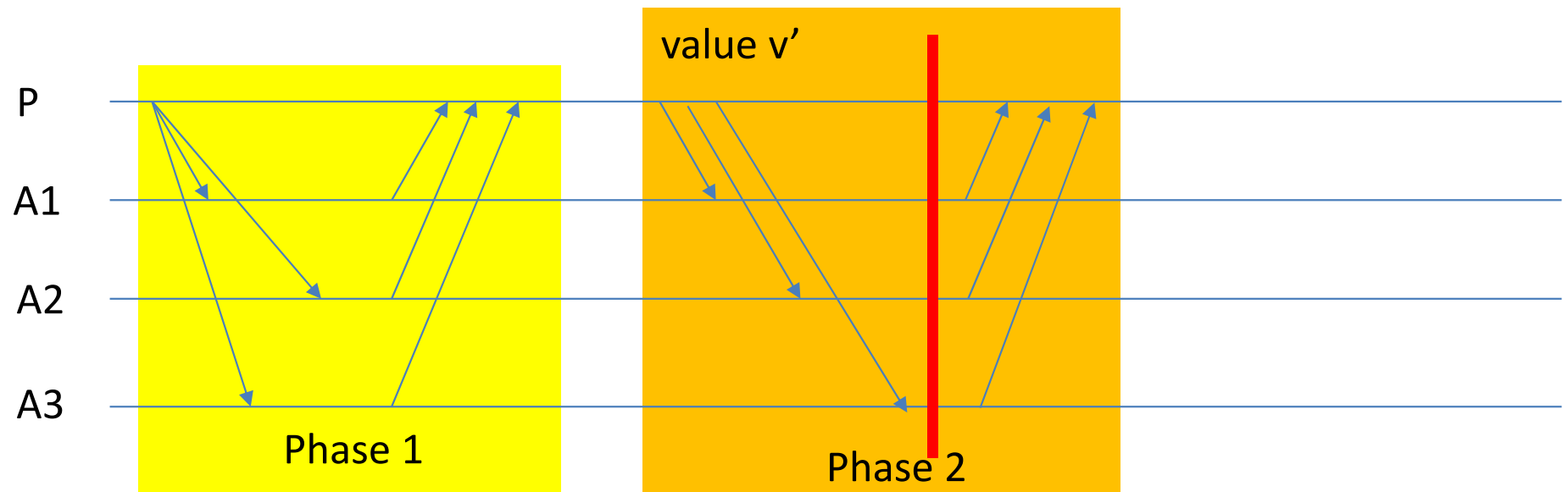
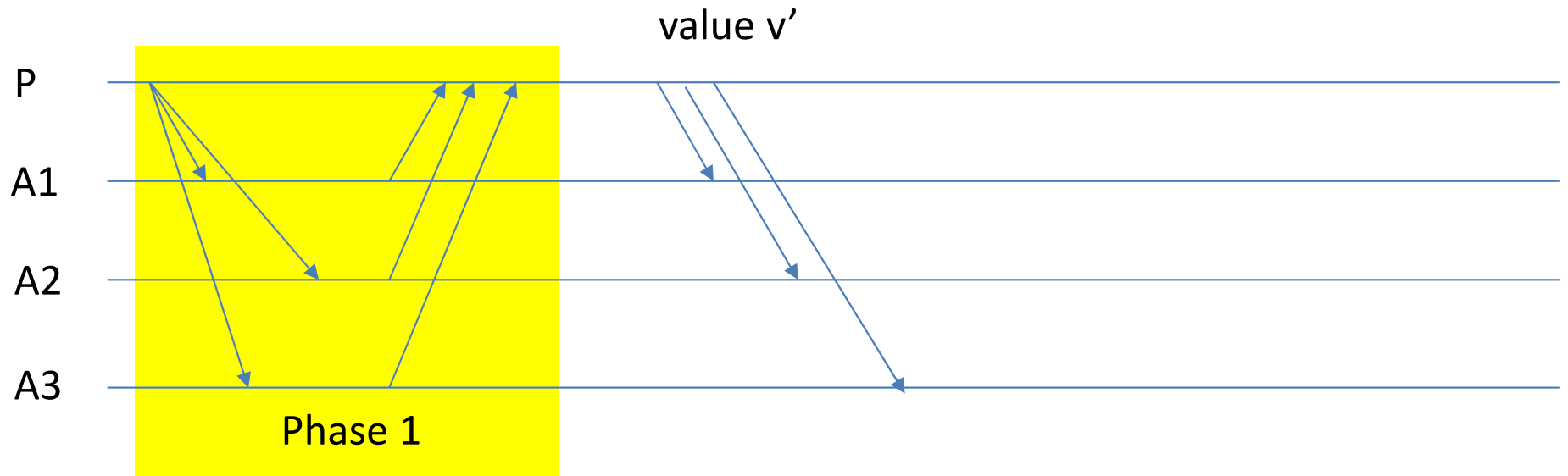
discussions

- If a majority of the acceptors have logs v on their disk, the value is chosen.
 - Why?



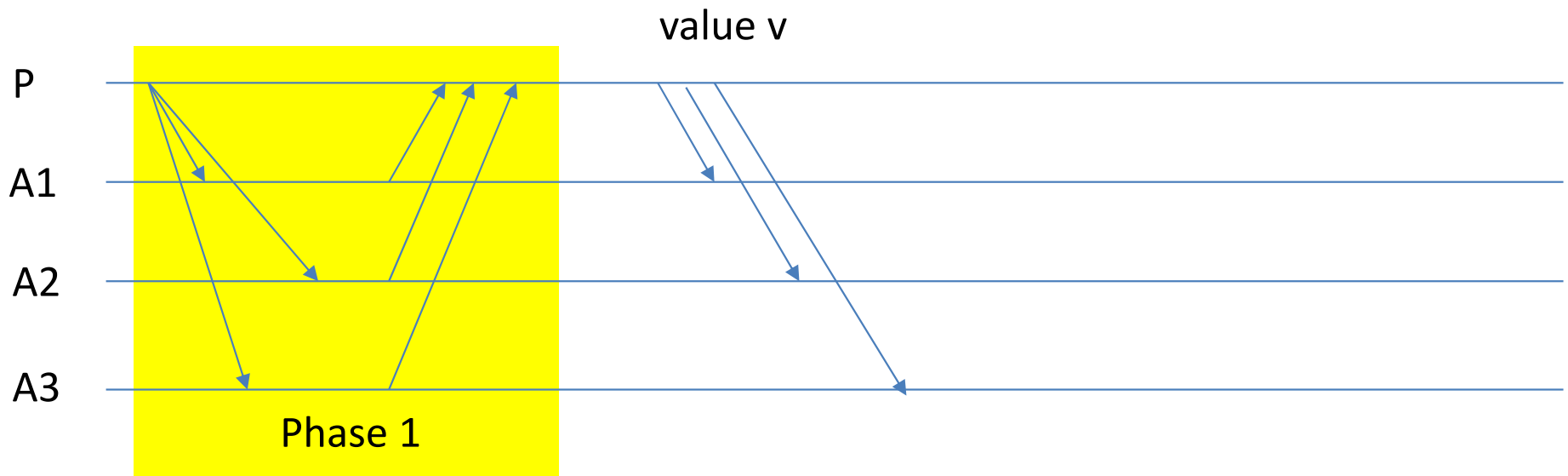


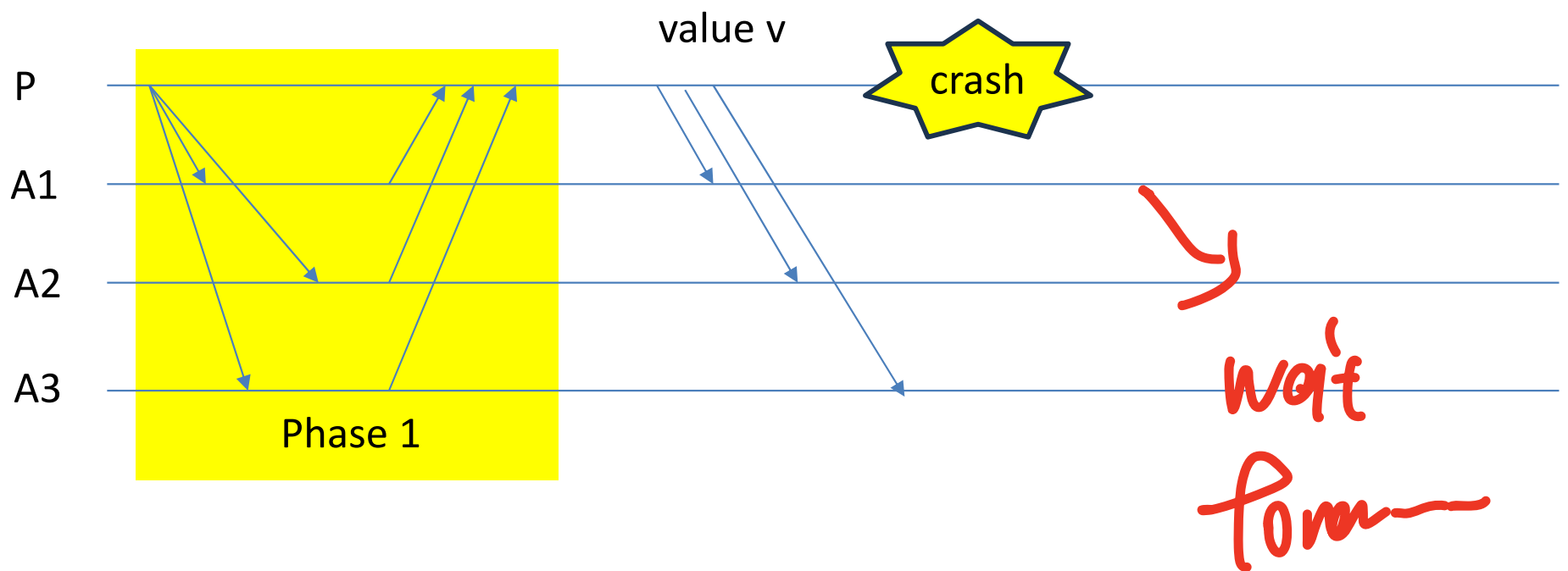
P2 must receive the ack from one of A2 and A3 to move to phase 2.



discussions

- After an acceptor receives the accept request of a proposer, can the acceptor ignore the prepare request from other proposers?
 - Why?



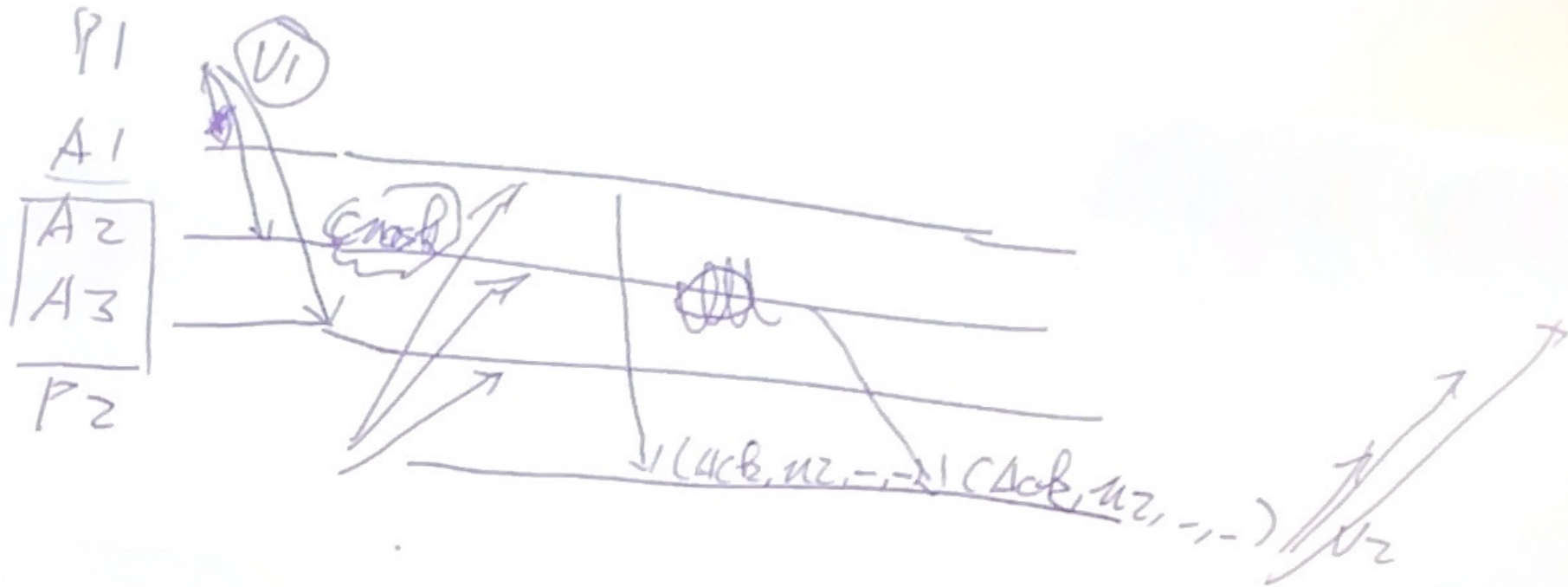


discussions

- In the response to the prepare request message, is it possible for the proposer to receive two ("ack", n , n' , v') and ("ack", n , n'' , v'') such that v' and v'' are not the same?

- P1 sends prepare request to A1, A2 and A3, and gets ack from all.
 - At phase 2, A1 receives v1, but A2 and A2 have not receive v1 yet.
- P2 sends prepare request to A1, A2, and A3. P2 receives A2's and A3's responds. But, not A1's response.
 - P2 starts phase 2 and send v2 to A1, A2 and A3. A2 and A3 receive v2. A1 has not received v2 yet.
- P3 sends prepare request to A1, A2 and A3. A1 response with v1 and A2 and A3 response with v2.
- Which value will P3 use in its accept request?
- Which value is the agreed value?

v2



- Which value will $P3$ use in its accept request?
- Which value is the agreed value?

Solution: log value in A_2

discussions

- The scheme can be optimised to reduce the number of messages
 - Have a single acceptor

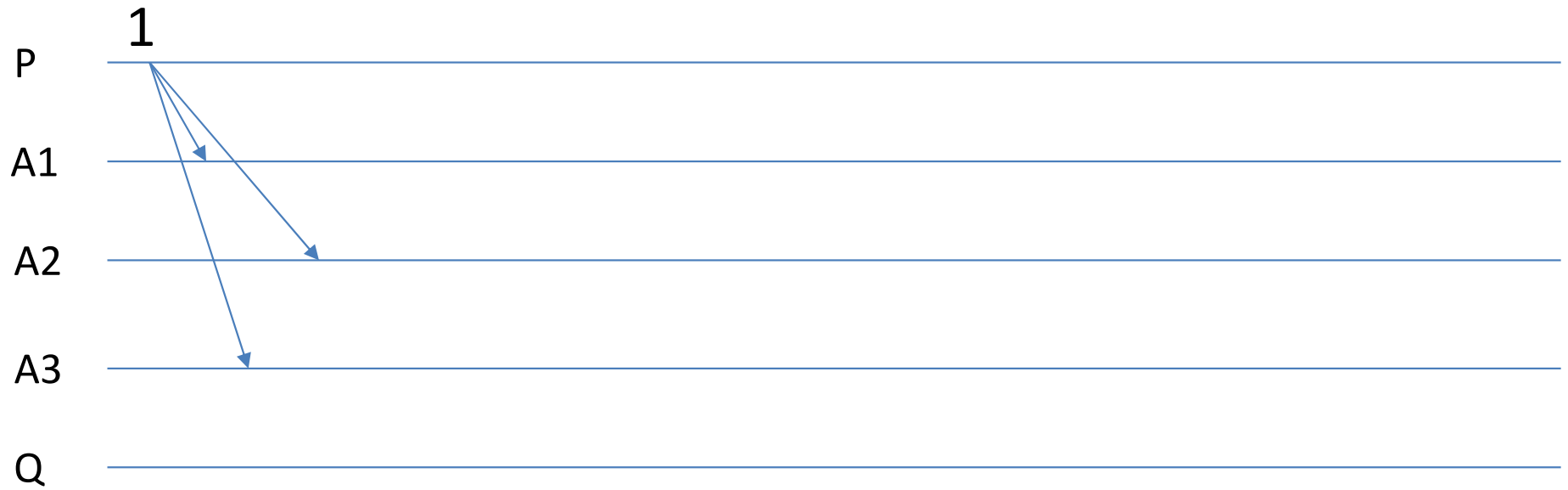
Safeness

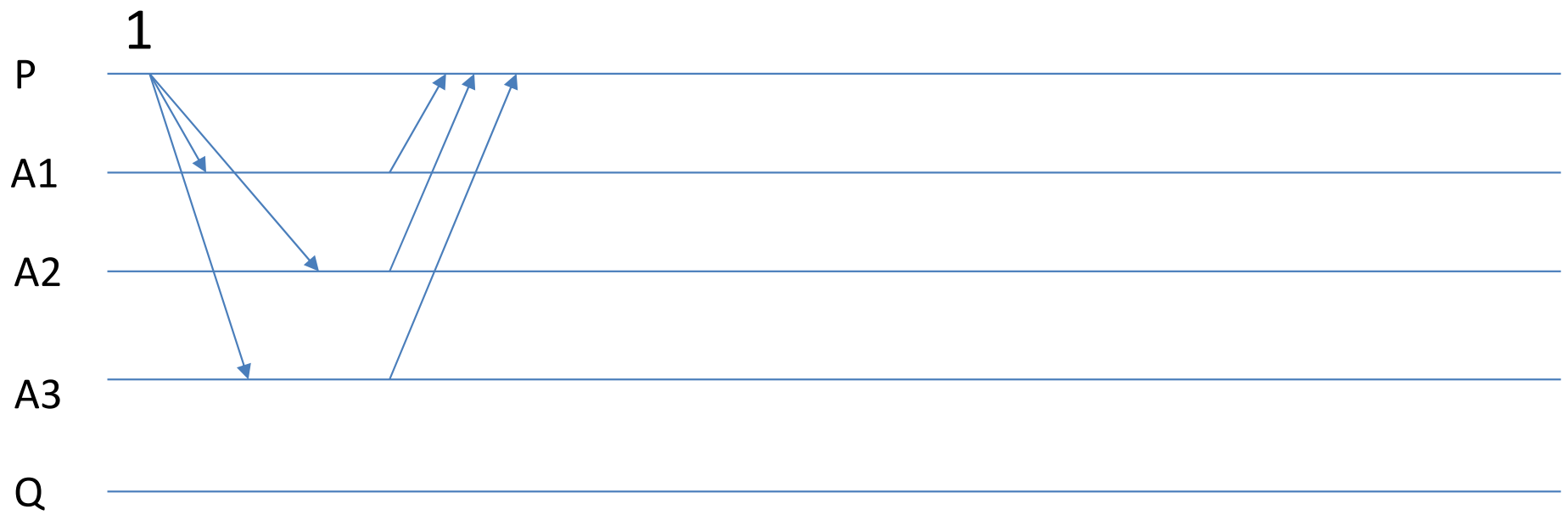
- As a value is **chosen** when a **majority** of the acceptors log v sent by the proposer in phase 2, if v in proposal (v, n) is chosen, the value in all accepted proposals (v', n') where $n' > n$ must satisfy $v = v'$
 - In the respond to the **prepare request**, **at least one** acceptor informs the proposer of the value that the majority acceptors have accepted.
 - The proposer **uses accepted value** with the largest version number as its own proposed value in the accept request.

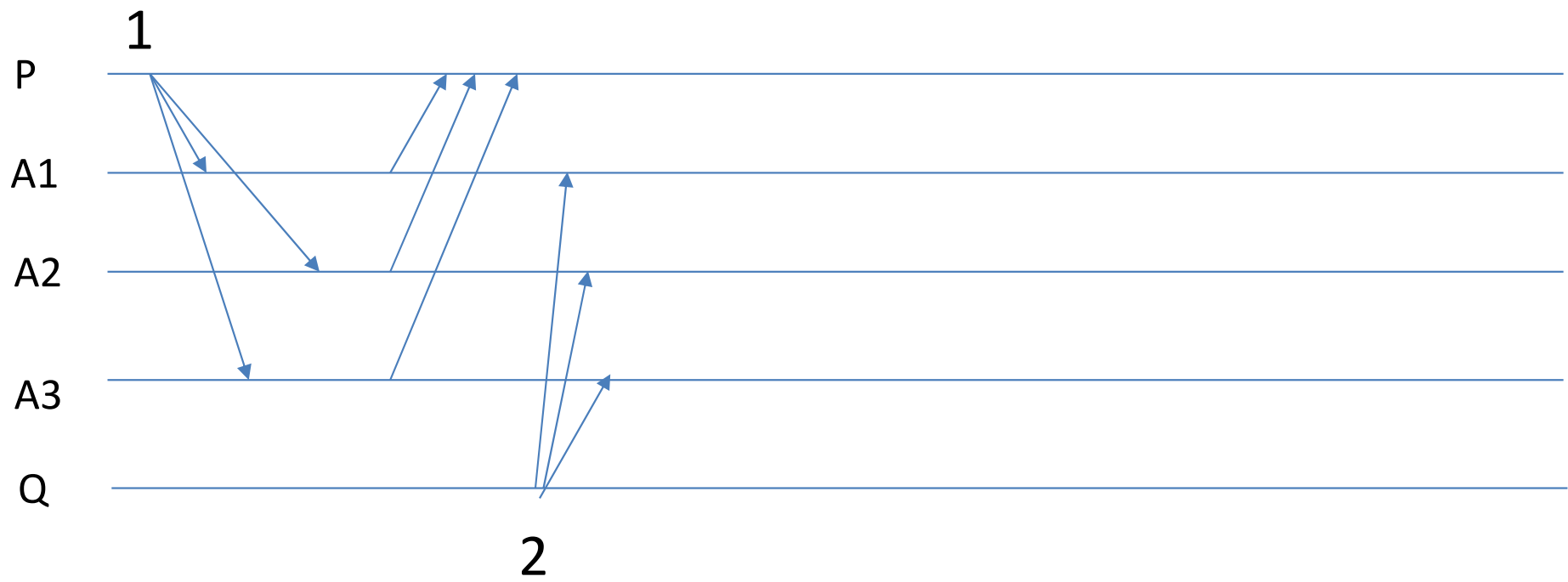
Liveness

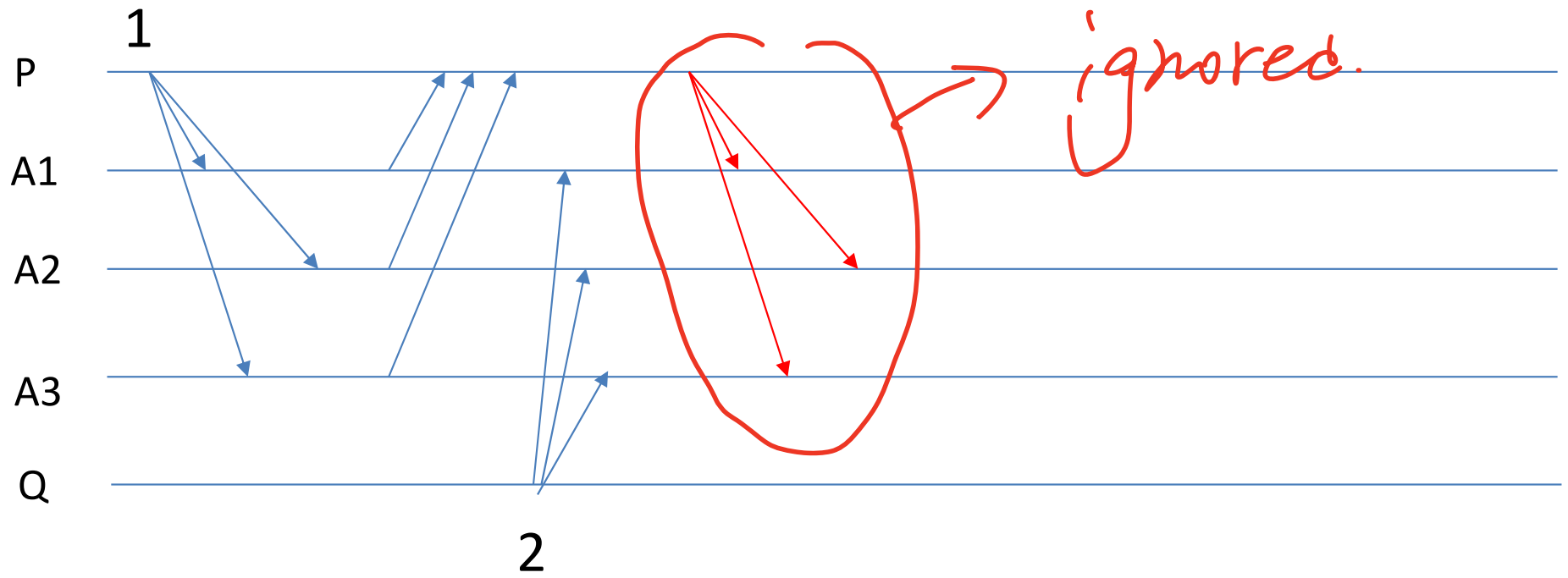
- Per FLP, Paxos cannot guarantee liveness.
- Proposer p completes phase 1 for a proposal number n_1 .
- Another proposer q then completes phase 1 for a proposal number $n_2 > n_1$.
- Proposer p 's phase 2 *accept* requests for a proposal numbered n_1 are ignored because the acceptors have all promised not to accept any new proposal numbered less than n_2 .
- Proposer p then begins phase 1 for a new proposal number $n_3 > n_2$, causing the second phase 2 *accept* requests of proposer q to be ignored.
- And so on.

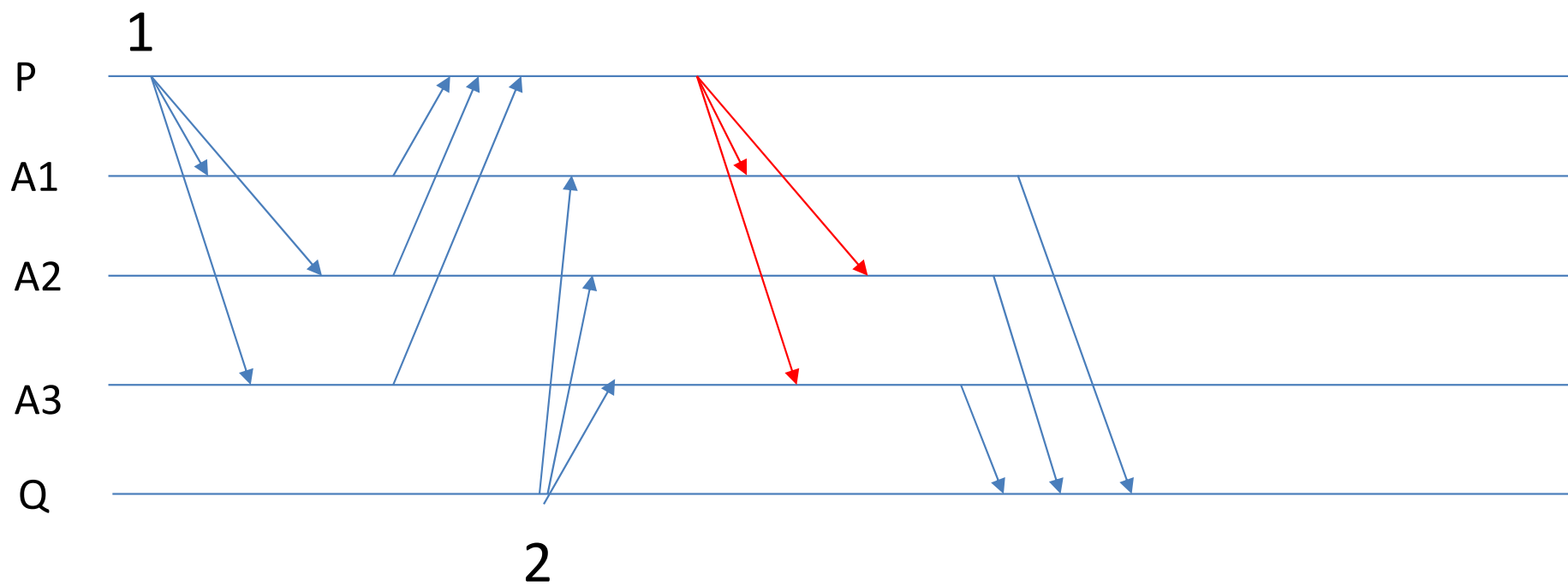
不停胎情况











Paxos vs Virtual Synchrony

- Ordering
 - Paxos enforces a strict global execution order of commands,
 - Virtual Synchrony only ensures agreement on the set of messages received within a view, without imposing an order.
- Failure Handling
 - Virtual Synchrony ensures messages received by at least one process are propagated to all surviving processes during a view change.
 - Paxos maintains a consistent log through quorum-based decision-making.
 - Once a decision is made, it is recorded by a majority of nodes, ensuring that after a failure, at least one surviving node will inform the leader of the decision, allowing the system to continue making progress without losing previously agreed-upon values.

Paxos in real life

- The replication services of some modern file systems uses Paxos
 - Google BigTable
 - Apache ZooKeeper, Yahoo
 - Many MS products, e.g. SQL server clusters

reviews

- Understand how FLP impossibility theorem affect real system design.
- Give a scenario in which the Paxos algorithm cannot terminate.
- What are the safety conditions of the Paxos algorithm?
- Paxos uses version numbers to help the acceptors to decide whether they should respond to a received prepare request message. Use an example to explain why version numbers are important.
 - hint: Without a version number, an acceptor needs to response to any prepare message. Consider whether you can find a scenario in which if two proposers send their prepare messages at the same time, none of them can get acknowledgement from a majority of the acceptors.

reviews

- In Paxos, what are the pros and cons of having a single acceptor?
- How does the Paxos algorithm guarantee that only the consensus value is propagated?
 - Hint: A decision is made when a majority of the acceptors have recorded the proposed value in the 2nd phase.
- Assume that a membership service that implements virtual synchrony is available and only one node can crash. Explain how the implementation of the Paxos algorithm can use the membership service to ensure the termination of the consensus algorithm.
 - Hint: Consider making the membership service to choose an unique proposer for each view.

Further readings

- Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. 1985. Impossibility of distributed consensus with one faulty process. *J. ACM* 32, 2 (April 1985), 374-382.
- Leslie Lamport. 1998. The part-time parliament. *ACM Trans. Comput. Syst.* 16, 2 (May 1998), 133-169.
- Lamport, Leslie (2001). Paxos Made Simple *ACM SIGACT News (Distributed Computing Column)* 32, 4 (Whole Number 121, December 2001) 51-58

- In the following discussion, assume that the version numbers of the three proposers, P1, P2 and P3, satisfy:
 - $P3's > P2's > P1's$
- P1 sends prepare request to A1, A2 and A3, and gets ack from all.
 - At phase 2, **A1 receives v1**, but **A2 and A2 have not receive v1 yet.**
- P2 sends prepare request to A1, A2, and A3. P2 **receives A2's and A3's responds**. But, **A1 has not received the prepare request message.**
 - P2 starts phase 2 and send **v2** to A1, A2 and A3. **A2 and A3 receive v2. A1 has not received v2 yet.**
- P3 sends prepare request to A1, A2 and A3. **A1 responses with v1** and **A2 and A3 response with v2.**
- **Which value will P3 use in its accept request?**
- **Which value is the agreed value?**

P1
A1
A2
A3
P2
P3

